Passo a passo da prática - Console AWS

Oct 14, 2026 · @Guilherme Rodrigues

Roteiro para montar ao vivo, no console da AWS, a API do Mural da Turma: DynamoDB → Lambda → API Gateway. Cada etapa termina com um checkpoint; só avance quando ele passar.

Antes de começar (2 min)

1. Entre no console com o usuário `gui-admin`, não com a root. A turma vai ver a tela.
2. No canto superior direito, escolha **N. Virginia (us-east-1)**. Todas as etapas precisam ficar na mesma região. A API reserva está em Ohio (us-east-2), então os nomes não colidem.
3. Deixe abertas três abas: DynamoDB, Lambda e API Gateway.
4. Deixe aberto num editor o arquivo `backend/lambda_function.py`, pronto para copiar.
5. Aumente o zoom do navegador para 125% ou 150%, para o console ficar legível no projetor.

Recurso	Nome que vamos usar
Tabela DynamoDB	`Recados`
Função Lambda	`mural-recados`
Política IAM	`AcessoRecados`
API	`mural-api`

Etapa 1 · Tabela no DynamoDB (5 min)

1. DynamoDB → **Tables** → **Create table**.
2. **Table name:** `Recados` (com R maiúsculo).
3. **Partition key:** `id`, tipo **String**. Deixe Sort key em branco.
4. Em **Table settings**, escolha **Customize settings** e, em **Read/write capacity settings**, marque **On-demand**.
5. Role até o fim e clique em **Create table**.

Checkpoint: o status muda de *Creating* para **Active**. Abra a tabela → **Explore table items**: está vazia.

Para falar: "Só declarei a chave. Não existe coluna autor nem texto: cada item traz os campos que quiser. On-demand significa que pagamos por leitura e escrita; parada, a tabela custa quase nada."

Etapa 2 · Função Lambda (10 min)

Criar

1. Lambda → **Create function** → **Author from scratch**.
2. **Function name:** `mural-recados`.
3. **Runtime: Python 3.13**. Se só aparecer uma versão mais nova, ela também serve.
4. **Architecture:** x86_64.
5. **Permissions:** deixe **Create a new role with basic Lambda permissions**. Essa role só permite escrever logs; a permissão da tabela vem na etapa 3.
6. **Create function**.

Colar o código

1. Na aba **Code**, abra `lambda_function.py`, apague tudo e cole o conteúdo de `backend/lambda_function.py`.
2. Clique em **Deploy** e espere a mensagem de sucesso. Sem Deploy, o teste roda o código antigo.

Variável de ambiente

1. **Configuration** → **Environment variables** → **Edit** → **Add environment variable**.
2. **Key:** `TABLE_NAME` · **Value:** `Recados`.
3. **Save**.

Primeiro teste (vai falhar, de propósito)

1. Aba **Test** → **Create new event**. **Event name:** `listar`.
2. Em **Event JSON**, apague o exemplo e cole:

```
{"rawPath": "/recados", "requestContext": {"http": {"method": "GET"}}}
```

3. **Save** e depois **Test**.

Resultado esperado: erro com `AccessDeniedException` e a mensagem "not authorized to perform: dynamodb:Scan".

Para falar: "A função existe e o código está certo, mas ela não tem permissão de ler a tabela. Na AWS, tudo começa negado. Quem libera é o IAM."

Etapa 3 · Permissão no IAM (5 min)

1. Na Lambda: **Configuration** → **Permissions** → em *Execution role*, clique no nome da role (`mural-recados-role-xxxx`). Abre o IAM numa nova aba.
2. **Add permissions** → **Create inline policy** → aba **JSON**.
3. Apague o conteúdo e cole:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "dynamodb:Scan",
      "dynamodb:PutItem",
      "dynamodb:DeleteItem",
      "dynamodb:UpdateItem"
    ],
    "Resource": "arn:aws:dynamodb:*:*:table/Recados"
  }]
}
```

4. **Next** → **Policy name:** `AcessoRecados` → **Create policy**.
5. Volte à aba da Lambda → **Test** → rode de novo o evento `listar`.

Checkpoint: `"statusCode": 200` e `"body": "[]"`. Se ainda der AccessDenied, espere 10 segundos e rode de novo: a permissão leva alguns segundos para valer.

Criar um recado pelo teste

1. **Test** → **Create new event** → **Event name:** `criar`.
2. Cole e rode:

```
{"rawPath": "/recados", "requestContext": {"http": {"method": "POST"}},
"body": "{\"autor\": \"Gui\", \"texto\": \"Primeiro recado!\"}"}
```

Checkpoint: `"statusCode": 201`. No DynamoDB → **Explore table items** → **Run**: o recado aparece, com `id`, `curtidas` e `criadoEm` gerados pela função.

Para falar: "A política libera só quatro ações, numa tabela só. Isso é privilégio mínimo. O atalho seria dar acesso total ao DynamoDB, e é assim que vazamentos começam."

Etapa 4 · API Gateway e rotas (10 min)

Criar a API

1. API Gateway → **Create API** → em **HTTP API**, clique em **Build**. Não é o REST API.
2. **Integrations** → **Add integration** → **Lambda** → região `us-east-1` → função `mural-recados`.
3. **API name:** `mural-api`. **IP address type:** IPv4. **Next**.

Configurar as rotas

O assistente cria sozinho uma rota `ANY /mural-recados`. Troque-a pelas quatro abaixo, todas com **Integration target** `mural-recados`:

Method	Resource path
`GET`	`/recados`
`POST`	`/recados`
`DELETE`	`/recados/{id}`
`POST`	`/recados/{id}/curtir`

Use **Add route** para as que faltarem e confira se a rota `ANY` sumiu. **Next**.

Stage e criação

1. Em **Define stages**, deixe `$default` com **Auto-deploy** ligado. **Next**.
2. Revise e clique em **Create**.
3. Na página da API, copie a **Invoke URL**, sem a barra final.

Checkpoint: no navegador, abra `<Invoke URL>/recados`. Aparece o recado criado na etapa 3, em JSON.

Para falar: "Sem o API Gateway, a Lambda não tem endereço na internet. Agora ela tem, e qualquer um com essa URL pode chamá-la. Repararam que ninguém pediu login? É isso que o Cognito resolveria."

Etapa 5 · CORS e teste no app (8 min)

Mostrar o erro primeiro

1. Rode o `flutter_solucao` no Chrome com a Invoke URL na `apiUrl`.

2. A lista carrega, mas **enviar** um recado falha com "Failed to fetch".
3. Aperte F12 → aba **Console**: aparece "blocked by CORS policy".

Para falar: "O GET simples passou porque a Lambda devolve o cabeçalho de CORS. O POST com JSON faz antes uma pergunta ao servidor, um OPTIONS, e nenhuma rota responde a ela. O navegador bloqueia por segurança. No celular isso não acontece: CORS é regra de navegador."

Corrigir

1. Na API → menu lateral **CORS** → **Configure**.
2. **Access-Control-Allow-Origin:** digite `*` e clique em **Add**.
3. **Access-Control-Allow-Headers:** digite `content-type` e clique em **Add**.
4. **Access-Control-Allow-Methods:** selecione `GET`, `POST`, `DELETE` e `OPTIONS`.
5. **Save**.

Com isso, o API Gateway responde sozinho ao OPTIONS e passa a ignorar os cabeçalhos de CORS que a Lambda devolve.

Checkpoint final: envie e apague um recado no app. Os dois funcionam sem erro no console do Chrome.

Passar para a turma

Cole a Invoke URL no chat da turma, como texto, e no slide "A API da turma". Lembre: **sem barra no final**.

Se algo der errado

Travou por mais de 5 minutos: passe para a turma a URL da reserva, `https://nk5fiyaei9.execute-api.us-east-2.amazonaws.com`, e continue depurando sem pressa.

Sintoma	Onde está o problema
Teste da Lambda dá `Runtime.ImportModuleError`	O arquivo não se chama `lambda_function.py`, ou faltou **Deploy**
`ResourceNotFoundException`	`TABLE_NAME` diferente do nome da tabela (maiúsculas contam) ou regiões diferentes
`AccessDenied` depois da etapa 3	Política colada na role errada, ou nome da tabela diferente de `Recados` no `Resource`

Sintoma	Onde está o problema
`{"message":"Not Found"}` no navegador	Rota não criada, caminho com erro de digitação, ou faltou `/recados` no fim da URL
`{"message":"Internal Server Error"}`	Erro dentro da Lambda: **Monitor** → **View CloudWatch logs** → log mais recente
`{"erro": "rota nao encontrada..."}`	A rota `ANY` do assistente ainda existe e mandou um caminho inesperado
CORS continua bloqueando	Esqueceu de clicar em **Add** depois de digitar o `*`, ou não clicou em **Save**

O erro dentro da Lambda é ótimo para ensinar: abra o CloudWatch na frente da turma e leia o log com eles.

Limpeza no fim da aula (5 min)

Faça na frente da turma, na ordem inversa da criação:

1. API Gateway → `mural-api` → **Actions** → **Delete**.
2. Lambda → `mural-recados` → **Actions** → **Delete**.
3. DynamoDB → `Recados` → **Actions** → **Delete table**.
4. IAM → **Roles** → `mural-recados-role-xxxx` → **Delete**.
5. CloudWatch → **Log groups** → `/aws/lambda/mural-recados` → **Actions** → **Delete**.

Para falar: "Na nuvem, o que fica ligado continua cobrando. Criar é metade do trabalho; desligar é a outra metade."

A reserva sai depois da aula, no PowerShell, com `terraform destroy` na pasta `backend/terraform`.